



本小节内容

fread 函数与 fwrite 函数

fgets 函数与 fputs 函数

fread 函数与 fwrite 函数

fread 函数与 fwrite 函数的具体形式如下：

```
int fread(void *buffer, size_t size, size_t num, FILE *stream);
```

```
int fwrite(const void *buffer, size_t size, size_t count, FILE *stream);
```

其中 buffer 是一个指针，对 fread 来说它是读入数据的存放地址，对 fwrite 来说它是输出数据的地址（均指起始地址）；size 是要读写的字节数；count 是要进行读写多少 size 字节的数据项；fp 是文件型指针；**fread** 函数的返回值是读取的内容数量，**fwrite** 写成功后的返回值是已写对象的数量。

下面来看例 1。

【例 1】fread 与 fwrite 的使用。

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char buf[20]="hello\nworld";
```

```
    FILE* fp;
```

```
    int ret;
```

```
    fp=fopen("file.txt","r+");//把这里改为 rb+后，再次执行程序观察，会发现文件大小是 11 个字节
```

```
    if(NULL==fp)
```

```
    {
```

```
        perror("fopen");
```

```
        return -1;
```



```
}
```

```
ret=fwrite(buf,sizeof(char),strlen(buf),fp); //把 buf 中的字符串写入文件
```

```
// ret=fread(buf,sizeof(char),sizeof(buf)-1,fp); //执行这个代码时, 需要注释上面的 fwrite, 因为
```

```
// puts(buf); //打印 buf 的内容
```

```
fclose(fp);
```

```
return 0;
```

```
}
```

fread 和 fwrite 函数既可以以文本方式对文件进行读写, 又可以以二进制方式对文件进行读写。

以"r+"即文本方式打开文件进行读写时, 向文件内写入的是字符串, 写完后右键文件, 选择属性 (如下图所示), 会发现大小是 12 个字节。



这是因为在文本方式下, 向文本文件中写入"\n"时实际存入磁盘的是"\r\n", 所有的接口调用都是 Windows 的系统调用, 这是 Windows 的底层实现所决定的 (Mac 和 Linux 不会)。当然,



以文本方式写入，一定要以文本方式读出，遇到"\r\n"时底层接口会自动转换为"\n"，因此用 fread 函数再次读取数据时，得到的依然是"hello\nworld"，共 11 字节。

如果把 fopen 函数中的"r+"改为"rb+"，也就是改为二进制方式，那么当我们向磁盘写入 11 字节时，磁盘实际存储的就是 11 字节，如果这时双击打开该文件，那么会发现没有换行，即 helloworld 是连在一起的，中间没有换行符，原因是 txt 文本编辑器必须遇到"\r\n"时才进行换行操作。



图 2 文件详细信息

相信读者此时已经理解了文本方式和二进制方式的差异（在 Mac 和 Linux 操作系统下并不存在这样的问题）。以文本方式下写入"\n"后，磁盘存储的是"\r\n"，当然读取时会以"\n"的形式读出"\r\n"。而以二进制方式写入"\n"后，磁盘存储的是"\n"。二者在其他方面没有差异。那么如何避免出错呢？如果是以文本方式写入的内容，那么一定要以文本方式读取；如果是以二进制方式写入的内容，那么一定要以二进制方式读取，不能混用！（大家主要理解文本文件与二进制文件的区别即可）。

```
#include <stdio.h>
```

```
int main(int argc, char** argv)
```



```
{  
  
    FILE* fp;  
  
    int i=123456;  
  
    int ret;  
  
    fp=fopen("file.txt","rb+");//以 rb+模式打开文件  
  
    if(NULL==fp)  
    {  
        perror("fopen");  
        return -1;  
    }  
  
    //向文件中写入整型数，如果我们双击打开文件会发现乱码，因为打开文件都是以字符格式去解析的  
  
    ret=fwrite(&i,sizeof(int),1,fp);  
  
    //ret=fread(&i,sizeof(int),1,fp);//打开本行注释及下面一行注释时，需要先注释 fwrite 代码  
  
    //printf("read ret=%d\n",ret);  
  
    fclose(fp);  
  
    return 0;  
}
```

上例中写入整型数，浮点数时，一定要用二进制方式时，需要以"rb+"方式打开文件，二进制方式下内存中存储的是什么，写入文件的就是什么，是一致的。例如，写入整型变量 i，其值为 123456，内存存储为 4 字节，即 0x0001E240，那么写入内存的也是 4 字节。这时双击打开文件看到的是乱码，所以读取时也要用一个整型变量来存储。



fgets 函数与 fputs 函数

函数 fgets 从给出的文件流中读取[num-1]个字符，并且把它们转储到 str（字符串）中。fgets 在到达行末时停止，fgets 成功时返回 str（字符串），失败时返回 NULL，读到文件结尾时返回 NULL。其具体形式如下：

```
char *fgets(char *str, int num, FILE *stream);
```

fputs 函数把 str（字符串）指向的字符写到给出的输出流。成功时返回非负值，失败时返回 EOF。其具体形式如下：

```
int fputs(const char *str, FILE *stream);
```

下面来看例 3。

【例 3】fgets 与 fputs 的使用。

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char buf[20]={0}; //用于存储读取数据
```

```
    FILE* fp;
```

```
    fp=fopen("file.txt","r+"); //打开文件
```

```
    if(NULL==fp)
```

```
    {
```

```
        perror("fopen");
```

```
        return -1;
```

```
    }
```

```
    while(fgets(buf,sizeof(buf),fp)!=NULL) //读取到文件结束时, fgets 返回 NULL
```

```
    {
```

```
        printf("%s",buf); //打印输出某一行内容
```

```
    }
```

```
return 0;  
  
}
```

使用 `fgets` 函数, 我们可以一次读取文件的一行, 这样就可以轻松地统计文件的行数。注意, 在做一些机试题目时 (机试是用 `fgets` 读标准输入, 因为 `gets` 部分学校机试不可用), 用于 `fgets` 函数的 `buf` 不能过小 (`buf` 大于最长行的长度), 否则可能无法读取 `"\n"`, 导致行数统计出错。`fputs` 函数向文件中写一个字符串, 不会额外写入一个 `"\n"`, 可以不用 `fputs`, 掌握 `fwrite` 即可。